

PRÁCTICA 5A – PRUEBAS DE SOFTWARE

Ingeniería del Software II

Mario Espinosa y Kiril Nazarenco.

2. Pruebas unitarias

2.1 Clase Cliente

Se han diseñado pruebas unitarias para la clase `Cliente`, excluyendo getters y setters. El método principal evaluado ha sido:

- `totalSeguros()`

Técnicas aplicadas:

- **Caja negra (partición equivalente):**
 - Cliente con seguros
 - Cliente sin seguros
 - Cliente con minusvalía / sin minusvalía

Ejemplo de caso de prueba:

- Cliente con dos seguros:
 - Seguro A: 100€
 - Seguro B: 150€
 - Resultado esperado: 250€

Resultado:

Las pruebas unitarias validan correctamente el cálculo del importe total de seguros del cliente.

2.2 Clase Seguro

También se han realizado pruebas unitarias sobre la clase `Seguro`, verificando:

- Correcta asignación de atributos
- Cálculo del precio del seguro (según potencia y cobertura)
- Comportamiento de getters y setters relevantes

Técnicas aplicadas:

- **Caja negra:**
 - Diferentes tipos de cobertura
 - Distintos valores de potencia
- **Caja blanca:**
 - Cobertura de decisiones en el cálculo del precio

3. Pruebas de integración

Se han implementado pruebas de integración sobre la clase `VistaAgente`, verificando la interacción entre:

- Capa de presentación (Swing)
- Capa de negocio (`GestionSeguros`)
- Capa de persistencia (DAO con H2 en memoria)

3.1 Casos de uso probados

Se ha validado el caso de uso **Consulta de Cliente**:

- Cliente existente en la base de datos
- Cliente sin seguros
- Cliente inexistente

3.2 Herramientas utilizadas

- JUnit 5
- AssertJ Swing
- Maven Failsafe Plugin
- Base de datos H2 en memoria

3.3 Estado inicial de la base de datos

Se utiliza una base de datos en memoria con datos precargados de clientes y seguros definidos en `H2ServerConnectionManager`.

4. Técnica de diseño de pruebas

Caja negra:

- Partición equivalente para DNI de cliente
- Casos válidos e inválidos de búsqueda

Caja blanca:

- Cobertura de sentencias en lógica de cálculo
 - Cobertura de decisiones en validación de cliente y seguros
-

6. Conclusión

Se ha conseguido implementar un conjunto de pruebas unitarias e integradas que verifican correctamente el funcionamiento del sistema. La integración entre capas ha sido validada mediante pruebas automatizadas, asegurando la correcta interacción entre GUI, lógica de negocio y base de datos en memoria.